

Контролисан развој уз AI

AI рад унутар архитектонских граница и проверљивог развојног поступка.

Услови за увођење AI алата

AI алати се уводе након успостављања архитектонских граница и тестирања функционалног језгра.

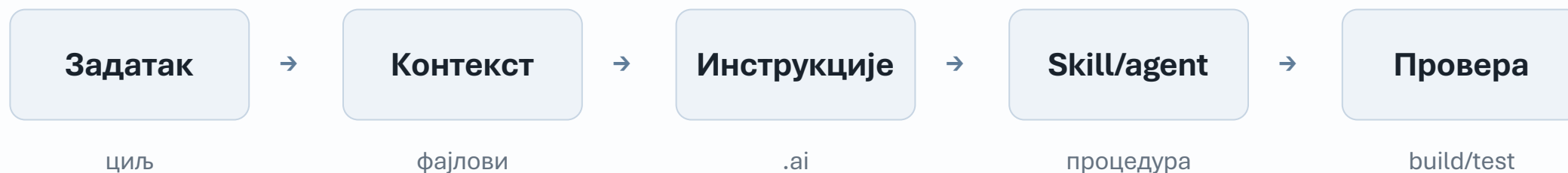
Циљ је да AI ради унутар граница solution-а и да сваки резултат остане проверљив стандардним развојним сигнаlima.

НАПОМЕНА

Резултати компилације, тестова и git diff-а представљају проверљиве доказе.

Контролисан AI радни ток

Задатак садржи циљ, контекст, ограничења и поступак провере.



НАПОМЕНА

AI радни ток је дефинисан инжењерски поступак.

Дефинисање проверљивог задатка

Задатак прецизира измену, дозвољени обим и критеријуме провере.

```
1 # Задатак
2 Једна резервација не сме тражити више од 5 комада опреме.
3 Не мењај код у овој фази.
4
5 # Ограничења
6 - Правило остаје у Domain/Application делу.
7 - API не сме садржати пословну одлуку.
8 - Постојећи тестови морају остати успешни.
```

НАПОМЕНЕ

Прво анализа, затим измена.

Ограничења чувају архитектуру.

План се прегледа пре дозволе за писање.

Структура AI задатка

Потпун задатак обухвата понашање, границе измене, доказе и ограничења.

01

Понашање

шта се мења у систему

02

Границе

који слојеви смеју да се мењају

03

Доказ

које команде и тестови потврђују резултат

04

Излаз

план, ризици, фајлови и сценарији

Верзионисане пројектне инструкције

Пројектна правила чувају се у датотеци `.ai/AI_INSTRUCTIONS.md`.

Инструкције нису општи савет, већ конкретна правила архитектуре solution-a: Domain не зависи од других пројеката, Application дефинише портове, Infrastructure имплементира port-ове.

НАПОМЕНА

AI_USAGE садржи сажет запис употребе и донетих одлука.

AI_INSTRUCTIONS.md као архитектонско правило

Документ наводи проверљива правила о зависностима, дозвољеном обиму и обавезним тестовима.

```
1  ## Архитектура
2  - Domain не зависи ни од једног другог пројекта.
3  - Application зависи само од Domain и дефинише port-ове.
4  - Infrastructure имплементира port-ове из Application
5    слоја.
6  - Api је composition root.
7  - MCP и guardrails не смеју постати зависност језгра.
```

НАПОМЕНЕ

Инструкције су део репозиторијума.

Тим их прегледа као код.

Модел не сме тврдити да је проверио нешто што није извршио.

Skill као поновљива процедура

Процедура за преглед Pull Request-а дефинише улазе, кораке и ограничења.

Review улога не мења код током review фазе, већ проверава захтев, diff, инструкције, тестове и архитектонске границе.

НАПОМЕНА

SRP се примењује и на улоге у AI радном току.

Поступак прегледа измене

Исти поступак примењује се на различите измене у solution-у.

- 01 Сажети очекивано понашање.
- 02 Проверити да ли diff излази из обима захтева.
- 03 Проверити Dependency Rule и границе модула.
- 04 Прегледати негативне и граничне сценарије.
- 05 Упоредити промењено понашање са тестовима.

Улоге и најмање привилегије

Свака улога добија само овлашћења потребна за конкретан задатак.

Улога	Дозволе	Ограничење
architecture-reviewer	чита solution, source, инструкције и diff	не пише код
implementer	мења фајлове из усвојеног плана	не проширује захтев
test/review	покреће тестове и анализира резултат	не мења тест да сакрије грешку

Структурирана предаја задатка агенту за имплементацију

План наводи дозвољене фајлове, очекиване измене и поступак провере.

```
1 {  
2   "task": "Implement approved reservation quantity rule",  
3   "constraints": [  
4     "Keep business rule outside Api",  
5     "Do not add Infrastructure dependency to  
6       Domain/Application"  
7   ],  
8   "filesToConsider": [  
9     "src/EquipmentReservation.Domain/InventoryItem.cs",  
10    "tests/EquipmentReservation.Tests/ReservationTests.cs"  
11  ]  
12 }
```

НАПОМЕНЕ

Фајлови су намерно ограничени.

Верификација је унапред дефинисана.

Ако се обим прошири, враћа се на анализу.

Провера AI резултата

Резултат се прихвата тек након компилације, тестова и прегледа diff-a.

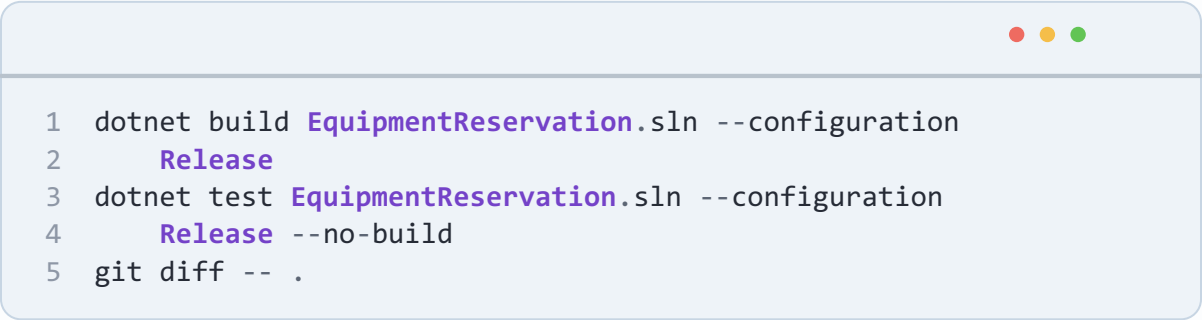
Завршетак задатка заснива се на стварним сигнаlima: успешан build, успешни NUnit тестови и преглед финалног diff-a.

НАПОМЕНА

Ако команда није извршена, не бележи се као успешна провера.

Независна провера измене

Команде за build, тестирање и diff обезбеђују независне сигнале.



```
1 dotnet build EquipmentReservation.sln --configuration
2   Release
3 dotnet test EquipmentReservation.sln --configuration
4   Release --no-build
5 git diff -- .
```

НАПОМЕНЕ

Пад теста се не решава брисањем очекивања.

Diff не сме да додирује неочекиване слојеве.

Резултат команде је јачи од уверења модела.

AI_USAGE као запис употребе

Документ садржи задатак, коришћени контекст, прихваћене одлуке и извршене провере.

01

Задатак

шта је тражено

02

Контекст

који фајлови и правила су коришћени

03

Одлука

шта је прихваћено или одбијено

04

Доказ

која провера је стварно извршена

Услови за прекид AI тока

Радни ток се прекида када су границе нарушене, тајне изложене или узрок грешке није објашњен.

УСЛОВИ ЗА ПРЕКИД

agent тражи .env или тајне

diff додирује неочекиване слојеве

тест пада без разумевања узрока

план проширује пословни захтев

УСЛОВИ ЗА НАСТАВАК

план је мали и проверљив

границе су испоштоване

тестови дају јасан сигнал

diff је разумљив за review

Одговорност тима

Тим прихвата измену тек након разумевања, провере и образложења одлука.

01

Разумевање

студент објашњава решење

02

Провера

команде и тестови доказују
понашање

03

Одлука

тим прихвата или одбија
предлог

Задатак на вежби

На копији EquipmentReservation solution-а изводи се једна контролисана измена уз AI алат.

- 01 Задати малу промену пословног правила.
- 02 Прво користити architecture-reviewer само за анализу.
- 03 Implementer-у проследити усвојен план.
- 04 Покренути build/test и прегледати diff.
- 05 Унети сажет запис у AI_USAGE.md.

Критеријум провере

Вредност AI алата процењује се на основу проверљивости резултата и очувања граница система.

3

стварна сигнала

Build, test и diff су минимум независне провере после AI измене.

Резиме

Контролисан AI радни ток примењује постојеће архитектонске и тестне механизме.

Када су контекст, ограничења, улоге и провера јасни, AI може помоћи без рушења SOLID и Clean Architecture правила.

НАПОМЕНА

Наредна вежба уводи MCP за контролисан приступ контексту.

Припрема за MCP

Идентификовати контекст и операције које MCP сме контролисано да изложи.

- 01 Идентификовати које информације AI сме само да чита.
- 02 Раздвојити resource од tool операција.
- 03 Одредити које команде смеју бити фиксно извршене.
- 04 Забележити шта никада не сме бити изложено као произвољан shell.
- 05 Понети build/test/diff сигнале као основу за MCP.